

KIN 318 “Programming and Data Science for Kinesiology”



Class introduction

Hyosub Kim, DPT, PhD / Associate Professor / School of Kinesiology

Land Acknowledgment



UBC's Vancouver Campus is located on the traditional, ancestral, and unceded territory of the Musqueam people. The land we are on now has been a place of learning for the Musqueam for thousands of years. We, as teachers and learners, have an obligation to uphold the best of that educational context here in the classroom and in our work.

Today's agenda

- Why this course?
- What will you learn this semester?
- Syllabus highlights
- Intro to course technologies (software platforms, programming language, etc.; also referred to as a *tech stack*) and some active learning!



Who is Dr. Hyosub (pronunciation: “Yo-sub”) Kim?



- Started at UBC in July 2023 – first time this class taught under this course number
- Director of [Computation, Cognition, and Movement Lab](#)
- Most enjoy teaching skills-based topics like programming/modeling/math/motor control
- Uses Python (programming language used for this course) a lot and wishes he had time to use it more!

Who is your TA?



- Jack Darley, 2nd year MSc student in CCM Lab
- Python pro
- Guitarist and sound engineer extraordinaire
- Ask him lots of questions!

Why this course?



Programming (coding) and data science are two of the most important skills a student can learn

- Generalizable - will help you no matter what area of science/industry/healthcare you enter
- You can immediately start contributing to your lab or team
- Helps to hone other skills (e.g., experimental design, statistics, modeling)
- Lots of testimonials from previous years' students re: how useful these skills have become

Bridges the gap between “traditional” computer science and data science courses

- Focus on practical computing skills
- Foundational coding skills (not always emphasized in introductory data science courses)
- Will learn to use the most popular data science code packages/libraries (e.g., NumPy, pandas, matplotlib, and seaborn)

Will prepare you for next stage of development

- Analyzing data in a lab
- Statistics/Machine learning
- Computational modeling

Motivating example



Parkinson's study of motor control

- We'll look at “raw” data first – painful!
- Making sense of tables of strings (collection of characters, representing text) and numbers with Python and Jupyter notebook
 - Show off skills you'll learn in this course

Syllabus and textbook



- Intro to syllabus
- Online textbook: <https://hyosubkim.github.io/datasci-for-kin>
 - Learning Objectives for this Course
 - Academic Integrity
 - Coding Tools:
 - Python – high-level language (i.e., more like natural language), free(!), open-source, must-know for data science/machine learning positions
 - **JupyterHub and Jupyter notebooks**
 - [DataCamp](#) – software platform for more practice; suggested due dates for assignments

Course format



- You are expected to complete readings and practice **before** coming to class – i.e., “flipped” classroom approach
- Short mini-lectures (10-15 minutes) during class
- Rest of the time is devoted to problems and practice – you can’t learn to code by listening to someone talk about it
- Think of your teaching team as knowledgeable and helpful guides
- Tutorial / Office hours: Thurs, 12 – 1:30, Gateway Health Building - B1401

How to succeed in this course



- Treat learning to code the same way you treat learning a sport, musical instrument, or any other skill – practice consistently, expect challenges, and keep at it
- Come to class
- Keep up with learning materials (readings, problem sets, etc.) – material is cumulative in this course
- You will be on your laptops for most classes – as a courtesy to other students, your instructional staff, and yourself, stay focused on what we're working on and don't check other apps/email/messages/etc.
- Ask questions as they come up, and come to tutorials!

More about this course



- Programming and data science are skills
- We welcome and invite questions, comments, ideas – we want everyone to put in the effort and succeed
- No experience necessary! Everyone in here is capable of learning this content *if* you prepare, practice, and ask questions
- Your exact starting point on the *total novice* → *expert** continuum does not matter—the goal is that we all learn together! [*technically, if you're anywhere close to expert, you shouldn't take this class.]
- Mistakes are inevitable and normal – the goal is to learn from them

Guidelines around GenAI



- One main goal of course is learning to write code yourself – to support you in achieving that goal you need to be able to learn and practice how to do this on your own. As such, use of GenAI to complete assessments (quizzes/midterm/final) is not permitted
- You have everything you need to learn how to code in this class without GenAI
- I recognize GenAI can be useful for certain tasks, and is now pervasive, but it can also be a huge distraction (especially when you are first starting to learn)
- Learn foundational concepts through course resources and your own effort – you will thank yourself later!
- More extensive explanations on Canvas in course syllabus

Guidelines around collaboration



- Make sure you've struggled on your own with a problem before asking for help. When it comes to coding and data analysis, not everything is mapped out. Learning inevitably involves cognitive struggle. That effortful feeling means you're getting somewhere!
- After you've tried on your own for a good while, take a break. Try again with a fresh mind. If you're completely stuck, you can ask a classmate for help by first explaining how you're approaching the problem.
- If you're being asked for help, do not provide a full answer or just show your code. Make sure the other party is explaining what they've tried and how they are approaching the problem. Provide a principle, point them to the relevant part of the textbook, ask a leading question, etc.
- Later, we'll go over pair programming (also part of your readings prior to Monday)

Course Breakdown



Midterms and Final Exam (90% of final grade)

- We are planning to use the new [Open Resource Centre for Assessments \(ORCA\)](#) – a computer-based testing facility with rooms at ICCS and Henning
- Provides a safe, equitable space to complete assessments
- Graded assessments will combine questions regarding basic coding knowledge, ability to interpret code, and writing code yourself

Coding Practice and Preparation (5%)

- Determined by on-time completion of assigned problem sets and/or lecture notebooks. These points are graded on a complete/incomplete basis.

Course Breakdown



In-Class Participation (5%)

- Throughout the semester, you will be asked to present your answers to problem sets, coding challenges, lecture notebooks, etc. individually and as part of a small group.
- This is not about correctness of answers, but demonstrating your thought process and for everyone to get comfortable with talking about code, which is a huge part of data science

Why [fill-in-the-blank] data?



- Data come from everywhere, in life and in this course
- Examples will include GDP, choice, neural, kinematic, physiological, [blank] data...*why?*
 - **(Realistic)** See first bullet point
 - **(Practical)** Being a good data scientist means you can approach any data set
 - **(Idealistic)** If you only work with data you are already familiar with, you will not grow as much

Intro to course tech stack



- Go to [Canvas page](#) – access JupyterHub accounts through course page link
 - When JH link first opens, choose “Python Notebook” under “Server Options”
- Use Firefox or Chrome as your default browser for this course—known issues with Safari
- Access JupyterHub and work through “lecture-00.ipynb” together – take 10 minutes to read on your own
 - Look in your “lectures” folder – that’s where future tutorial/lecture notebooks will be as well
- Make sure everyone can access Assignment 0
- **Jupyter notebook versions of the textbook chapters are in your JupyterHub**

About UBC Jupyter Open



- Amazing resource that allows us to coordinate content for entire course BUT it may sometimes be slow and/or buggy
- In such cases, you can try to:
 - Wait a minute and refresh
 - Log in through private window (Incognito)
- If problems persist, contact **both** Learning Technology (LT.hub@ubc.ca, or visit the LT Hub online: <https://lthub.ubc.ca/dropin/>) and your TA to let us know – LT HUB is typically very responsive

Before next class session (Mon, 9/14)



- Read syllabus
- Read first two chapters of [online textbook](#) (“About this course”, “Introduction to Data Science”)
- Complete A0 by Sunday 11:59 pm and submit to Canvas
 - Register for PrairieLearn and ORCA/PrairieTest plus several other tasks
- Start going through Chapter 3 (“Intro to Python”)
 - You can read the textbook version that is on the web OR you can go through the Jupyter notebook versions of textbook in your JupyterHub accounts (“textbook-chapters” folder)
 - Nice to look at web version at first, just to get an idea of content. Soon after, start interactively engaging by going through notebook versions.

The Zen of Python

Beautiful is better than ugly.

Explicit is better than implicit.

Simple is better than complex.

Complex is better than complicated...

Readability counts...

In the face of ambiguity, refuse the temptation to guess...

If the implementation is hard to explain, it's a bad idea.

If the implementation is easy to explain, it may be a good idea.

- Tim Peters

(type “import this” in a code cell to read the whole poem)

